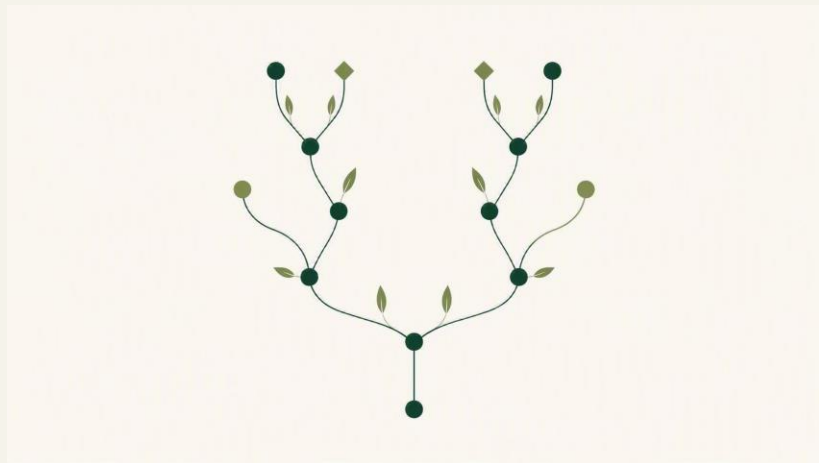


Árboles de Decisión y Algoritmos de Ensamble

De un solo árbol a los bosques y el boosting: cómo funcionan, cómo se visualizan y cuándo elegir cada uno.

Nivel técnico-intermedio · Scikit-learn, XGBoost, LightGBM, CatBoost



Lo que vamos a recorrer

01

Fundamentos del árbol

Nodos, divisiones, Gini vs. entropía y overfitting

02

Visualización con Scikit-learn

plot_tree, export_graphviz y export_text

03

Métodos de ensemble

Bagging vs. boosting: dos filosofías

04

Random Forest y Extra Trees

Muchos árboles que votan

05

Gradient Boosting moderno

XGBoost, LightGBM y CatBoost

06

Comparativa y guía práctica

Rendimiento, interpretabilidad y elección

01

FUNDAMENTOS

El árbol de decisión

El bloque de construcción de todos los modelos basados en árboles.

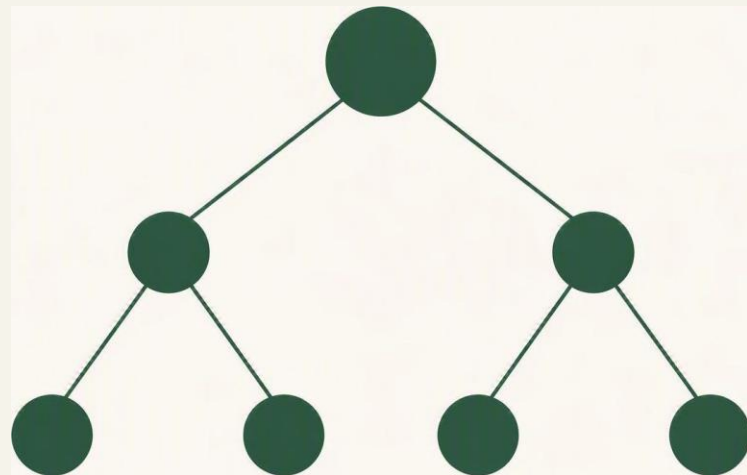
CONCEPTO

¿Qué es un árbol de decisión?

Un modelo que clasifica (o predice) haciendo una serie de preguntas encadenadas sobre las variables.

Como un juego de “20 preguntas”: cada corte divide los datos y reduce la incertidumbre hasta llegar a una predicción.

- **Raíz** — El primer corte, sobre la variable más informativa
- **Nodos internos** — Cada uno aplica una condición del tipo variable $\leq$ umbral
- **Hojas** — El final de una rama: entregan la clase o el valor predicho



Raíz → nodos → hojas

Cómo se construye, corte a corte

1

Evaluar cortes

Para cada variable prueba posibles umbrales de división

2

Medir la impureza

Elige el corte que deja los grupos más “puros” (Gini/entropía)

3

Dividir y repetir

Aplica el mejor corte y continúa de forma recursiva

4

Detenerse

Para al alcanzar hojas puras o un criterio de parada

El árbol es voraz (greedy): en cada nodo toma la mejor decisión local, sin mirar hacia adelante.

Gini vs. Entropía: medir la impureza

Ambas miden qué tan “mezcladas” están las clases en un nodo. El árbol elige el corte que más reduce la impureza.

Impureza de Gini

$$\text{Gini} = 1 - \sum p_i^2$$

- Probabilidad de clasificar mal una muestra al azar
- Rápida de calcular: es el valor por defecto en scikit-learn
- Tiende a aislar la clase más frecuente

Entropía / Ganancia de info.

$$H = - \sum p_i \cdot \log_2(p_i)$$

- Mide el desorden en bits; usa logaritmos
- Un poco más costosa por el cálculo del log
- Resultados casi idénticos a Gini en la práctica

En la mayoría de los casos la elección entre Gini y entropía apenas cambia el árbol final.

EJEMPLO

Un árbol leyendo Edad e Ingreso

Con dos variables, el árbol encadena umbrales hasta asignar una clase (“Sí” / “No”) en cada hoja.

- **Primer corte: Edad $\leq$ 31.5**
- Si es menor, predice directamente “No”
- **Si es mayor, mira Ingreso $\leq$ 50.000**
- El ingreso alto inclina la hoja hacia “Sí”

```
export_text()  
  
| --- Edad <= 31.5  
| | --- class: No  
| --- Edad > 31.5  
| | --- Ingreso <= 50000  
| | | --- class: No  
| | --- Ingreso > 50000  
| | | --- class: Sí
```

Overfitting: cuando el árbol memoriza

Un árbol sin límites crece hasta clasificar perfecto el entrenamiento... incluido el ruido. Luego generaliza mal con datos nuevos.

Síntoma

Accuracy casi perfecto en entrenamiento y bajo en validación

Causa

Árbol demasiado profundo con hojas de muy pocas muestras

Remedio

Podar: limitar profundidad, exigir mínimos por hoja o usar ensambles

Regla clave: un árbol individual es interpretable pero inestable — pequeños cambios en los datos producen árboles muy distintos.

Hiperparámetros que domestican el árbol

Hiperparámetro	Qué controla	Efecto práctico
<code>max_depth</code>	Profundidad máxima del árbol	Menor = más simple, menos overfitting
<code>min_samples_split</code>	Mínimo de muestras para dividir un nodo	Valores altos frenan cortes ruidosos
<code>min_samples_leaf</code>	Mínimo de muestras por hoja	Suaviza y estabiliza las predicciones
<code>max_features</code>	Variables candidatas por corte	Introduce aleatoriedad (clave en ensambles)
<code>criterion</code>	Gini o entropía	Métrica de impureza para elegir el corte
<code>ccp_alpha</code>	Poda por complejidad de costo	Recorta ramas que aportan poco

02

SCIKIT-LEARN

Visualizar el árbol

La mejor forma de entender cómo fue entrenado un árbol es verlo.

MÉTODO RECOMENDADO

plot_tree(): el árbol de un vistazo

Dibuja el árbol directamente con Matplotlib. Es la opción más simple y la más recomendable para aprender.

- Muestra la pregunta de cada nodo
- Colorea las clases con filled=True
- Ideal para explicar el modelo en clase

```
plot_tree()

from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 8))
plot_tree(
    modelo,
    feature_names=["Edad", "Ingreso"],
    class_names=["No", "Sí"],
    filled=True)
plt.show()
```

Qué muestra cada nodo en `plot_tree`

1

Condición

La pregunta del corte, p. ej. $\text{Edad} \leq 31.5$

2

Impureza

El valor de Gini o entropía del nodo

3

`samples`

Cuántas muestras llegaron a ese nodo

4

`value`

La distribución de clases dentro del nodo

5

`class`

La clase predicha (dominante) en la hoja

6

Color

Con `filled=True`: tono según la clase mayoritaria

export_graphviz(): para documentar

Exporta el árbol al formato Graphviz (.dot) y produce diagramas nítidos y vectoriales, ideales para informes y presentaciones.

- Salida escalable de máxima calidad
- Personalizable: colores, rótulos, redondeo
- Requiere la librería Graphviz instalada

```
export_graphviz()  
  
from sklearn.tree import  
    export_graphviz  
  
export_graphviz(  
    modelo,  
    out_file="arbol.dot",  
    feature_names=["Edad", "Ingreso"],  
    class_names=["No", "Sí"],  
    filled=True, rounded=True)
```

SIN GRÁFICO

export_text(): el árbol como texto

Cuando no querés un gráfico, imprime la estructura del árbol en la consola. Perfecto para logs, tests o revisiones rápidas.

- Sin dependencias de graficación
- Fácil de versionar y comparar
- Muestra umbrales y clase por rama

```
export_text()

from sklearn.tree import export_text

print(export_text(
    modelo,
    feature_names=["Edad", "Ingreso"]))
```

Salida en consola

```
|--- Edad <= 31.5 → No
|--- Edad > 31.5
|   |--- Ingreso > 50000 → Sí
```

Tres formas de ver un árbol

`plot_tree()`

Gráfico rápido

La más sencilla y visual.
Recomendada para aprender y explicar.

★ *Para aprender*

`export_graphviz()`

Máxima calidad

Diagramas vectoriales nítidos para documentación y presentaciones.

Para documentar

`export_text()`

Solo texto

Estructura en consola, sin dependencias gráficas. Ideal en pipelines.

Para inspeccionar

Para entender cómo funciona un árbol, `plot_tree()` es la opción más recomendable.

03

ENSAMBLES

De un árbol a muchos

Combinar modelos débiles para obtener uno fuerte y estable.

Bagging vs. Boosting

Casi todos los modelos potentes de datos tabulares combinan árboles. La diferencia está en cómo los combinan.

Bagging · en paralelo

- Muchos árboles independientes entrenados a la vez
- Cada uno ve una muestra distinta de los datos
- Se combinan por votación o promedio
- **Reduce la varianza → Random Forest, Extra Trees**

Boosting · secuencial

- Árboles entrenados uno tras otro
- Cada árbol corrige los errores del anterior
- Aprendizaje guiado por el gradiente del error
- **Reduce el sesgo → XGBoost, LightGBM, CatBoost**

04

BAGGING EN ACCIÓN

Random Forest

En lugar de confiar en un solo árbol, crea cientos y los hace votar.

LA IDEA CENTRAL

La fuerza está en el número

Un único árbol puede equivocarse porque aprendió demasiado ruido del entrenamiento. Random Forest lo resuelve con diversidad.

“En lugar de confiar en un solo árbol, crea cientos de árboles y hace que voten.”

Más diversidad entre árboles → predicciones más robustas.



Dos dosis de aleatoriedad

Para que cada árbol sea distinto, Random Forest hace dos cosas al entrenar:



Bootstrap (muestra de datos)

Cada árbol se entrena con un subconjunto aleatorio de los registros, extraído con reemplazo. Ninguno ve exactamente los mismos datos.



Subconjunto de variables

En cada división considera solo un subconjunto aleatorio de variables (p. ej. 4–5 de 20), no todas. Así descorrelaciona los árboles.

Resultado: cientos de árboles que cometen errores diferentes e independientes entre sí.

Votación por mayoría

¿Comprará el cliente? Cada árbol emite su voto y la clase más votada gana.

Árbol 1

Compra

Árbol 2

No compra

Árbol 3

Compra

Árbol 4

Compra

Árbol 5

No compra

Recuento

3 Compra

2 No compra

Predicción final: **Compra**

En regresión, en vez de votar, se promedian las predicciones de todos los árboles.

¿POR QUÉ FUNCIONA MEJOR?

Los errores se cancelan entre sí

Un árbol individual puede equivocarse por haber aprendido ruido del entrenamiento. Pero al combinar cientos de árboles entrenados de forma diferente, esos errores tienden a compensarse: los aciertos de unos cubren los fallos de otros.

Más preciso

que un árbol individual

Más estable

ante cambios en los datos

Menos overfitting

por promediar la varianza

Random Forest: ventajas e hiperparámetros

Por qué es un gran modelo base

- Preciso y robusto casi sin ajuste
- Resistente al overfitting
- Maneja bien variables y escalas mixtas
- Entrega importancia de variables
- **Limitación: menos interpretable que un árbol**

Hiperparámetros clave

<code>n_estimators</code>	Número de árboles del bosque
<code>max_features</code>	Variables candidatas por corte
<code>max_depth</code>	Profundidad de cada árbol
<code>min_samples_leaf</code>	Mínimo de muestras por hoja
<code>n_jobs</code>	Paralelización del entrenamiento

Extra Trees: aún más aleatoriedad

Extremely Randomized Trees lleva la idea de Random Forest un paso más allá al elegir los umbrales de corte al azar.

En qué se diferencia

- No busca el mejor umbral: lo elige al azar
- Mayor aleatoriedad → árboles más descorrelacionados
- Suele usar todo el dataset (sin bootstrap por defecto)

Cuándo brilla

- Muy rápido de entrenar
- A veces iguala o supera a Random Forest
- Baja aún más la varianza del modelo

05

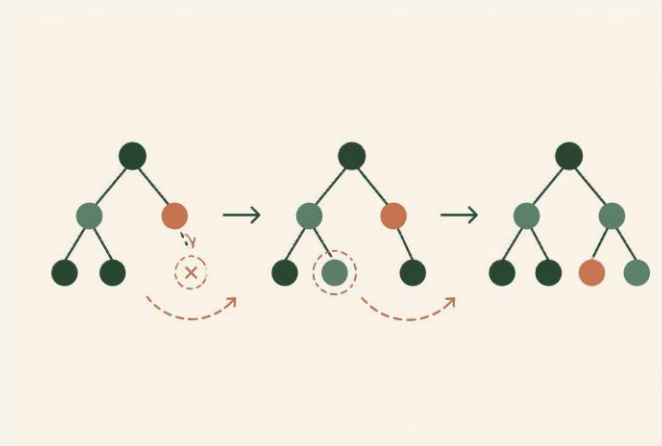
BOOSTING

Gradient Boosting

Árboles secuenciales donde cada uno corrige los errores del anterior.

Aprender de los errores, árbol a árbol

- Se construyen árboles de forma secuencial
- Normalmente son árboles poco profundos (weak learners)
- Cada árbol se entrena sobre los residuos del anterior
- El error restante guía el siguiente árbol (gradiente de la pérdida)
- El ensemble se fortalece con cada iteración



Corrige el error → suma el árbol → repite

A diferencia del bagging, aquí el orden importa: el boosting reduce el sesgo, no solo la varianza.

Boosting pide más ajuste

Generalmente supera a Random Forest, pero a cambio expone más hiperparámetros que hay que calibrar con cuidado.

learning_rate	Cuánto aporta cada árbol	Bajo = más preciso pero necesita más árboles
n_estimators	Número de árboles / iteraciones	Se equilibra con el learning rate
max_depth	Profundidad de cada árbol	Controla la complejidad de cada corrección
subsample	Fracción de datos por iteración	Añade aleatoriedad y regulariza
reg_lambda / reg_alpha	Regularización L2 / L1	Penaliza la complejidad y frena el overfitting

XGBoost: el estándar de oro

Durante años fue el referente en competencias de Machine Learning sobre datos tabulares. Arregló la escalabilidad y la regularización del boosting clásico.

Regularización nativa

L1 y L2 integradas para controlar el overfitting

Muy preciso

Rendimiento de referencia en tareas tabulares

Maneja valores faltantes

Aprende la mejor dirección para los NaN

Ecosistema maduro

Documentación, comunidad y soporte enormes

Contrapartida: es el que más hiperparámetros pide afinar de los tres modernos.

LightGBM: velocidad y escala

Desarrollada por Microsoft, resolvió la velocidad de entrenamiento de XGBoost en datos de alta dimensión. Muy usada en producción.

Crecimiento leaf-wise

Expande la hoja más prometedora, no por niveles

Histogram-based

Agrupar valores en bins para acelerar los cortes

Extremadamente rápido

Ideal para datasets grandes y muchas variables

Bajo consumo de memoria

Escala bien donde otros se quedan cortos

A vigilar: el crecimiento leaf-wise puede sobreajustar en datasets pequeños si no se limita.

CatBoost: rey de las categóricas

Diseñada alrededor de las variables categóricas y del problema de fuga de información (target leakage) que arrastraban los otros.

Categóricas nativas

Sin one-hot manual: las procesa internamente

Ordered boosting

Evita el sesgo por fuga del target

Poco preprocesamiento

Buenos resultados casi con los valores por defecto

Muy competitivo

A la altura de XGBoost y LightGBM en precisión

Elección natural cuando el dataset tiene muchas variables categóricas de alta cardinalidad.

CARA A CARA

XGBoost · LightGBM · CatBoost

	XGBoost	LightGBM	CatBoost
Origen	DMLC	Microsoft	Yandex
Foco	Robustez y control	Velocidad y escala	Categóricas y sesgo
Crecimiento	Por nivel	Por hoja (leaf-wise)	Árboles simétricos
Categóricas	Requiere codificar	Soporte integrado	Nativo y óptimo
Velocidad	Media–alta	Muy alta	Alta
Ajuste típico	El que más pide	Moderado	El que menos pide

Los tres son gradient boosting: no compiten como opciones arbitrarias, cada uno resolvió un límite del anterior.

06

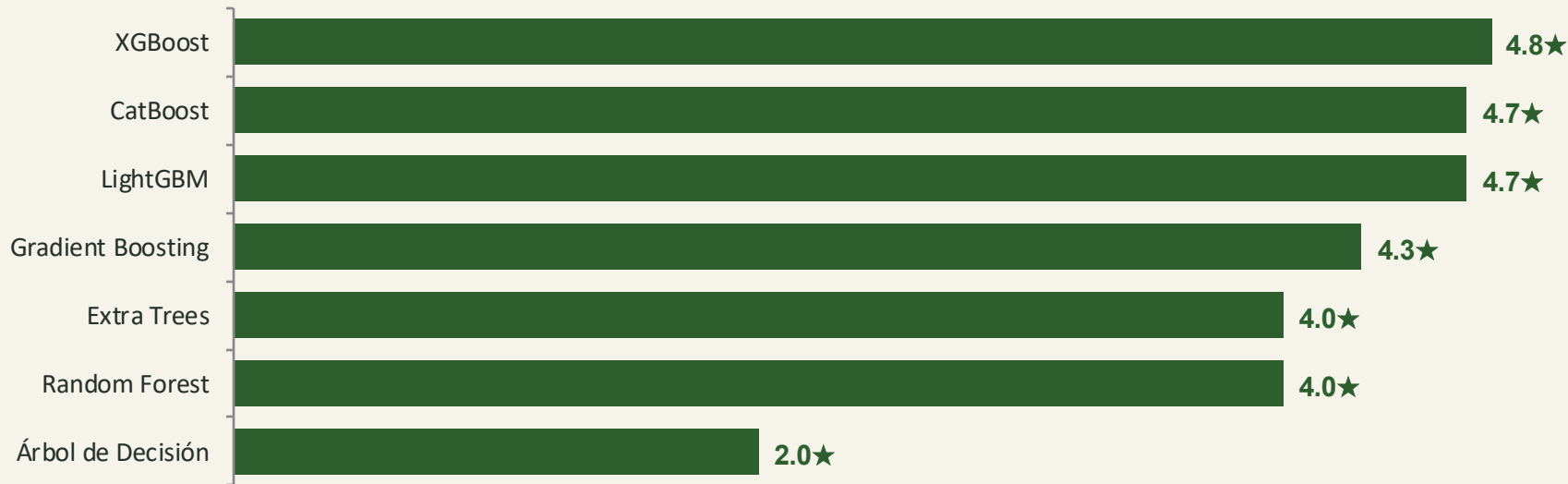
PANORAMA

Comparativa y elección

Precisión, interpretabilidad y la pregunta final: ¿cuál elijo?

Rendimiento típico en datos tabulares

No existe un ranking universal: depende de los datos y del problema. Aun así, un orden promedio de peor a mejor suele verse así.



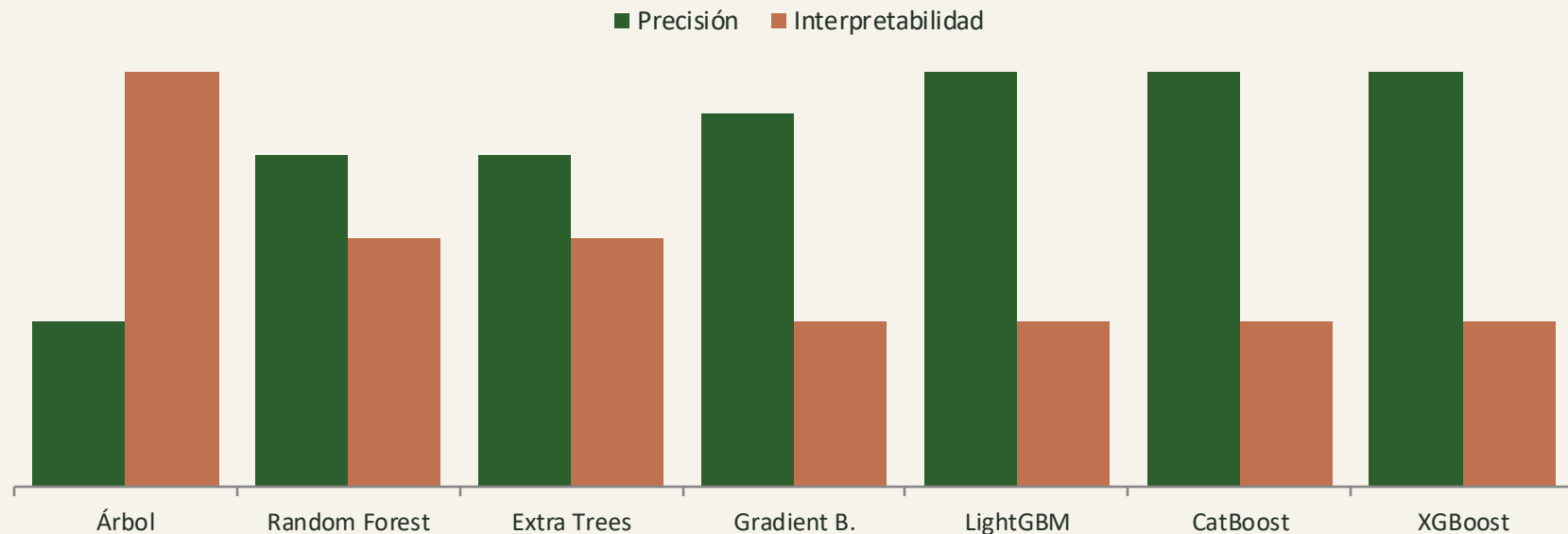
Escala orientativa sobre 5 (★). Los modelos de boosting encabezan; el árbol individual queda al final por su tendencia al overfitting.

Precisión frente a interpretabilidad

Algoritmo	Precisión	Interpretabilidad
Árbol de Decisión	★★	★★★★★
Random Forest	★★★★	★★★
Extra Trees	★★★★	★★★
Gradient Boosting	★★★★½	★★
LightGBM	★★★★★	★★
CatBoost	★★★★★	★★
XGBoost	★★★★★	★★

Existe un trade-off claro: a mayor precisión, menor transparencia del modelo.

Cuando ganás precisión, perdés claridad

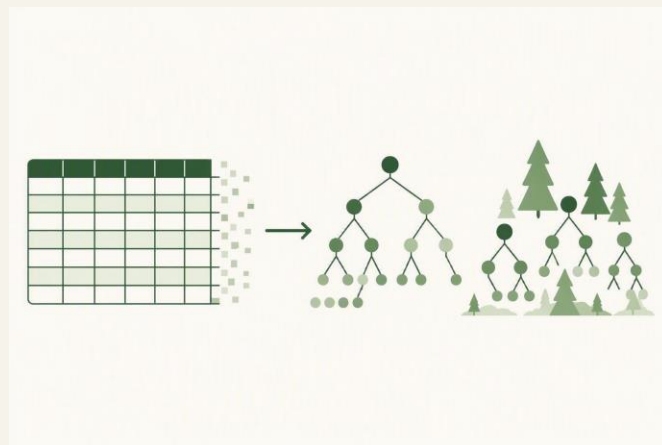


Verde: precisión · Terracota: interpretabilidad — las barras se cruzan al movernos de izquierda a derecha.

Por qué los árboles dominan lo tabular

Aunque el deep learning reina en imágenes, texto y audio, los modelos de árboles siguen siendo el estándar en datos tabulares.

- **Fronteras de decisión irregulares**
- Cortes por umbral que capturan relaciones no lineales
- **Invariancia de escala**
- No requieren normalizar las variables
- **Manejo de faltantes y tipos mixtos**
- Toleran datos heterogéneos del mundo real



>80% de los problemas de negocio reales
usan datos tabulares

¿Qué algoritmo elijo?

SI QUERÉS

Para enseñar conceptos

→ **Árbol de Decisión**
Interpretable y fácil de visualizar

SI QUERÉS

**Modelo sólido sin mucho
ajuste**

→ **Random Forest**
Preciso y robusto casi sin tuning

SI QUERÉS

**Máximo rendimiento
tabular**

→ **XGBoost · LightGBM · CatBoost**
El podio cuando la precisión manda

Ideas clave

1

Un árbol enseña, un bosque produce El árbol es transparente pero inestable; los ensambles ganan precisión y robustez.

2

Visualizar es entender `plot_tree`, `export_graphviz` y `export_text` revelan cómo decide el modelo.

3

Bagging vs. boosting Random Forest reduce varianza en paralelo; el boosting reduce sesgo en secuencia.

4

El boosting lidera lo tabular XGBoost, LightGBM y CatBoost marcan el techo de precisión hoy.

5

Elegí según el objetivo Claridad, robustez sin ajuste o máxima precisión: no hay un único ganador.

GRACIAS

De un árbol a un bosque: interpretabilidad y potencia en el mismo terreno.

Árboles de Decisión · Random Forest · Gradient Boosting · XGBoost · LightGBM · CatBoost